

**School of Engineering**

Course : Diploma in Electronic & Computer Engineering (EGDF20)

Module : EGE322 – IoT System Project

Laboratory 8: IoT Dashboard using MQTT with Adafruit IO**1. Introduction**

Adafruit IO is a cloud-based Internet of Things (IoT) platform that allows users to collect, monitor, store, and visualize sensor data through an online dashboard. It supports communication protocols such as MQTT and REST API, making it suitable for IoT applications using microcontrollers such as the ESP32.

In this laboratory exercise, the ESP32 microcontroller will be connected to a DHT22 temperature and humidity sensor. The sensor data will be transmitted to the Adafruit IO cloud platform using MQTT communication and displayed on dashboard widgets such as gauges and charts.

This lab also introduces the concept of:

- Uplink communication (ESP32 → Cloud)
- Downlink communication (Cloud → ESP32)

Students will also learn how to remotely control an LED connected to the ESP32 from the Adafruit IO dashboard through MQTT subscribe communication.

Adafruit IO provides both free and paid cloud services. The free version supports basic IoT applications with limitations on the number of dashboards, feeds, and data rates, while the paid version provides additional features and higher usage limits.

This laboratory exercise provides hands-on experience in cloud-based IoT communication, MQTT publish-subscribe systems, dashboard visualization, and remote device control commonly used in modern IoT applications.

2. Procedures

Setting up an Adafruit.io account

- Head over to io.adafruit.com and click Sign In to log into IO using your Adafruit account. It's free and fast to join.
- You will be directed to your profile page showing your plan, connected services, and account data:

The screenshot shows the Adafruit.io website's sign-in and order status section. The top navigation bar includes links for Shop, Learn, Blog, Forums, LIVE!, AdaBox, and IO. The main content area is divided into two columns. The left column, titled 'SIGN IN', contains a form with fields for 'EMAIL OR USERNAME' and 'PASSWORD', a 'SIGN IN' button, and a link to 'NEED AN ADAFRUIT ACCOUNT? SIGN UP'. The right column, titled 'ORDER STATUS', contains a form with fields for 'EMAIL ADDRESS' and 'ORDER NUMBER', a 'CHECK ORDER STATUS' button, and a link to 'Where do I find this?'. The Adafruit logo is visible in the top left corner.

- Click Sign-in

Create a Dashboard

- Click '+Dashboard' to add a new dashboard.

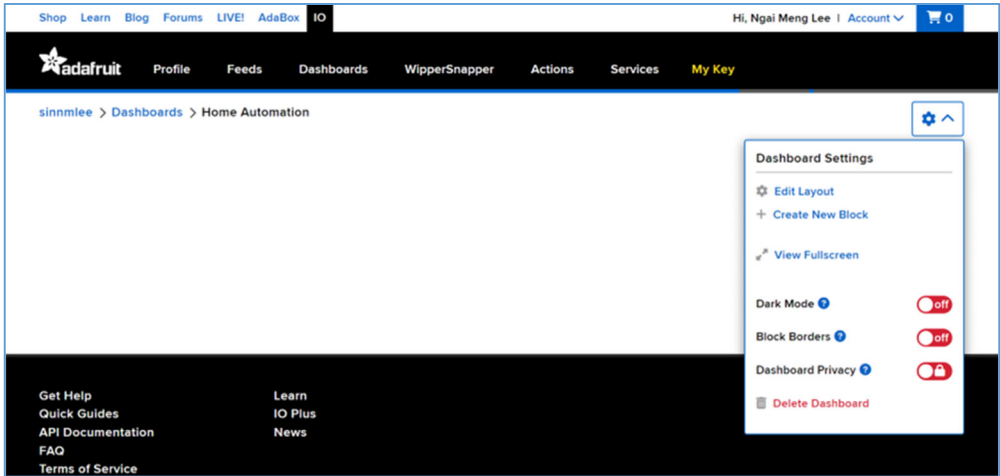
The screenshot shows the 'Dashboards' page on the Adafruit.io website. The top navigation bar includes links for Shop, Learn, Blog, Forums, IO, LIVE!, and AdaBox. The main content area shows the user's profile 'CFCFLEE3' and a list of dashboards. A red box highlights the '+ New Dashboard' button. A red box also highlights the 'Dashboards' tab in the top navigation bar. A green hand icon points to the 'Dashboards' tab.

The screenshot shows the 'Create a new Dashboard' modal form. The form has two input fields: 'Name' and 'Description'. The 'Name' field contains the text 'Home Automation' and the 'Description' field contains the text 'Home Automation Dashboard'. There are 'Cancel' and 'Create' buttons at the bottom of the modal. The background shows the user's profile 'sinmlee' and a list of dashboards.

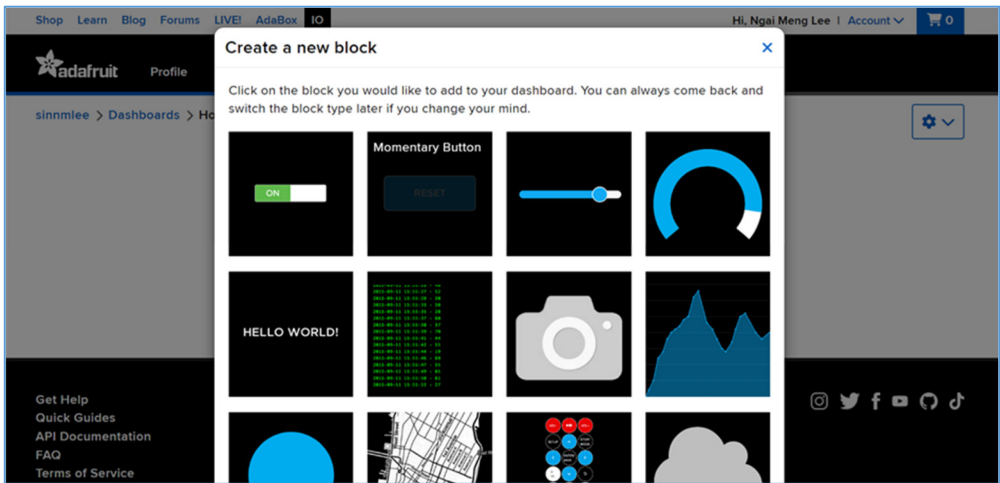
- Click Create.

Create New Block or Widget

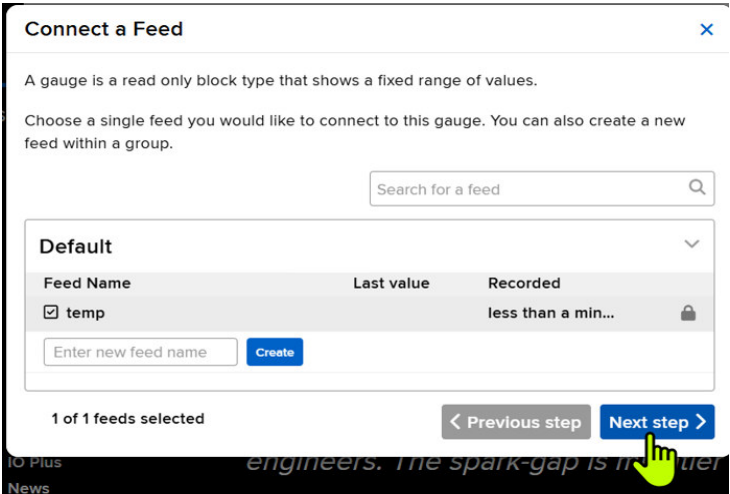
f) Click 'Create New Block'.



g) Select and add the widget that you like. (We use Gauge here)



h) Create a new feed temp.



i) Click 'Next Step'

Block settings

In this final step, you can give your block a title and see a preview of how it will look. Customize the look and feel of your block with the remaining settings. When you are ready, click the "Create Block" button to send it to your dashboard.

Block Title (optional)

Temperature

Gauge Min Value

0

Gauge Max Value

100

Gauge Width

25px

Gauge Label

deg C

Low Warning Value

Block Preview

27

deg C

0

100

Gauge A gauge is a read only block type that shows a fixed range of values.

Test Value

Enter the properties you would like to display.

j) You can go to edit layout to edit the properties of the dashboard.

sinmlee > Dashboards > Home Automation

Temperature

0

deg C

0

100

Dashboard Settings

Edit Layout

Create New Block

View Fullscreen

Dark Mode

Block Borders

k) Add in another widget for **humidity (hum)** by following previous adding "Temperature" (Temp) steps.

CFCFLEE3 / Feeds

New Feed

New Group

Default

Feed Name	Key	Last value	Recorded
☐ hum	hum		6 minutes ago
☐ light	light	OFF	1 minute ago
☐ temp	temp		10 minutes ago

Lab 8. IoT Dashboard using MQTT with Adafruit IO

Page 4 of 10

Effective Date: 2026 S1

- l) Add in a switch to toggle the 'Light'

Block settings

In this final step, you can give your block a title and see a preview of how it will look. Customize the look and feel of your block with the remaining settings. When you are ready, click the "Create Block" button to send it to your dashboard.

Block Title (optional)

Block Preview


Button On Text

Limit of 6 characters for the toggle text. Use the block title to be more descriptive.

Button On Value (uses On Text if blank)

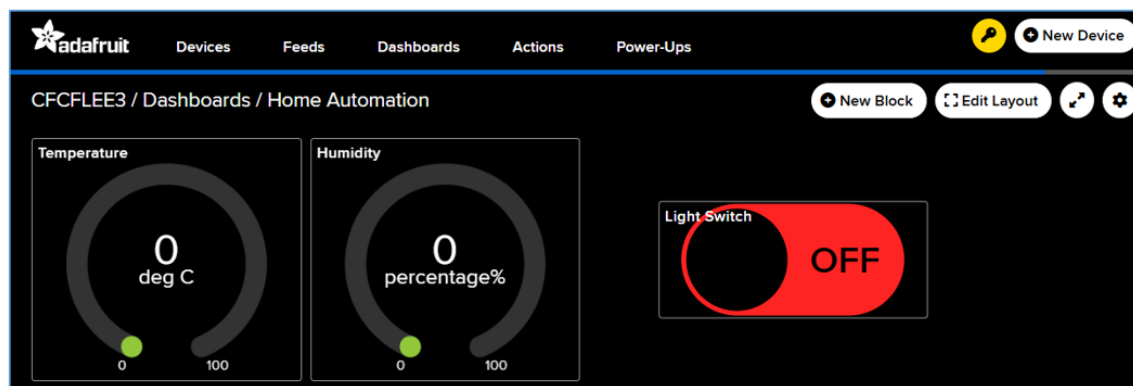
Button Off Text

Limit of 6 characters for the toggle text. Use the block title to be more descriptive.

Toggle A toggle button is useful if you have an ON or OFF type of state. You can configure what values are sent on press and release.

Enter the values of 1 and 0 for Lights ON and OFF and text to display when Light is ON and OFF.

- m) Click 'Create Box'.



- n) Edit the layout to reposition the widgets as you like it.
- o) Click to Save Layout.

Create the micropython script

- p) After setting up the Adafruit.io Dashboard, let us take a look at the micropython script how to publish and subscribe data to/from the Adafruit.io using MQTT.

```
# The MIT License (MIT)
# Copyright (c) 2019 Mike Teachman
# https://opensource.org/licenses/MIT
# Modified by CF
from machine import Pin
import network
import time
from umqtt.robust import MQTTClient
import os
import gc
import sys
import dht
import credential

sensor = dht.DHT22(Pin(16)) # read temp and humidity
led=Pin(4,Pin.OUT) # turn on LED with switch

# the following function is the callback which is
# called when subscribed data is received
#def cb(topic, msg):
#    print('Subscribe: Received Data: Topic = {}, Msg = {}'.format(topic, msg))
#    state=int(str(msg,'utf-8'))
#    if state==1:
#        led.value(1)
#    else:
#        led.value(0)

def cb(topic, msg):
    print('Subscribe: Received Data: Topic = {}, Msg = {}'.format(topic, msg))
    try:
        message = msg.decode('utf-8').strip().lower()

        if message in ['1', 'on', 'true']:
            led.value(1)
        else:
            led.value(0)
    except Exception as e:
        print("Error in callback:", e)
```

```

# WiFi connection information
WIFI_SSID = credential.ssid
WIFI_PASSWORD = credential.password

# turn off the WiFi Access Point
ap_if = network.WLAN(network.AP_IF)
ap_if.active(False)

# connect the device to the WiFi network
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect(WIFI_SSID, WIFI_PASSWORD)

# wait until the device is connected to the WiFi network
MAX_ATTEMPTS = 20
attempt_count = 0
while not wifi.isconnected() and attempt_count < MAX_ATTEMPTS:
    attempt_count += 1
    time.sleep(1)

if attempt_count == MAX_ATTEMPTS:
    print('could not connect to the WiFi network')
    sys.exit()

#debug
print("Connected to WiFi. IP:", wifi.ifconfig()[0])
# create a random MQTT clientID
random_num = int.from_bytes(os.urandom(3), 'little')
mqtt_client_id = bytes('client_'+str(random_num), 'utf-8')

# connect to Adafruit IO MQTT broker using unsecure TCP (port 1883)
#
# To use a secure connection (encrypted) with TLS:
# set MQTTClient initializer parameter to "ssl=True"
# Caveat: a secure connection uses about 9k bytes of the heap
# (about 1/4 of the micropython heap on the ESP8266 platform)
ADAFRUIT_IO_URL = b'io.adafruit.com'
ADAFRUIT_USERNAME = credential.adafruit_username
ADAFRUIT_IO_KEY = credential.adafruit_io_key
ADAFRUIT_IO_FEED1 = b'temp'
ADAFRUIT_IO_FEED2 = b'hum'
#ADAFRUIT_IO_FEED3 = b'Lights'

```

```

ADAFRUIT_IO_FEED3 = b'light'

client = MQTTClient(client_id=mqtt_client_id,
                    server=ADAFRUIT_IO_URL,
                    user=ADAFRUIT_USERNAME,
                    password=ADAFRUIT_IO_KEY,
                    ssl=False)

try:
    client.connect()
except Exception as e:
    print('could not connect to MQTT server {}'.format(type(e).__name__, e))
    sys.exit()

# publish free heap statistics to Adafruit IO using MQTT
# subscribe to the same feed
#
# format of feed name:
# "ADAFRUIT_USERNAME/feeds/ADAFRUIT_IO_FEEDNAME"
mqtt_feedname = bytes('{s}/feeds/{s}'.format(ADAFRUIT_USERNAME,
ADAFRUIT_IO_FEEDNAME), 'utf-8')
temp_feed = bytes('{s}/feeds/{s}'.format(ADAFRUIT_USERNAME,
ADAFRUIT_IO_FEED1), 'utf-8')
hum_feed = bytes('{s}/feeds/{s}'.format(ADAFRUIT_USERNAME,
ADAFRUIT_IO_FEED2), 'utf-8')
mqtt_feed = bytes('{s}/feeds/{s}'.format(ADAFRUIT_USERNAME,
ADAFRUIT_IO_FEED3), 'utf-8')

client.set_callback(cb)
client.subscribe(mqtt_feed)
PUBLISH_PERIOD_IN_SEC = 10
SUBSCRIBE_CHECK_PERIOD_IN_SEC = 0.5
accum_time = 0
while True:
    try:
        # Publish
        if accum_time >= PUBLISH_PERIOD_IN_SEC:
            sensor.measure() # Measuring
            temp = sensor.temperature() # getting Temp
            #debug
            print(temp)
            hum = sensor.humidity()
            #debug

```



```

print(hum)

client.publish(temp_feed,
               bytes(str(temp), 'utf-8'), # Publishing Temp feed to adafruit.io
               qos=0)
client.publish(hum_feed,
               bytes(str(hum), 'utf-8'), # Publishing Temp feed to adafruit.io
               qos=0)
#free_heap_in_bytes = gc.mem_free()
#print('Publish: freeHeap = {}'.format(free_heap_in_bytes))
#client.publish(mqtt_feedname,
               #bytes(str(free_heap_in_bytes), 'utf-8'),
               #qos=0)
accum_time = 0

# Subscribe. Non-blocking check for a new message.
client.check_msg()
time.sleep(SUBSCRIBE_CHECK_PERIOD_IN_SEC)
accum_time += SUBSCRIBE_CHECK_PERIOD_IN_SEC
except KeyboardInterrupt:
    print('Ctrl-C pressed...exiting')
    client.disconnect()
    sys.exit()

```

- q) You will need to create your own credential.py file for your password and access code.

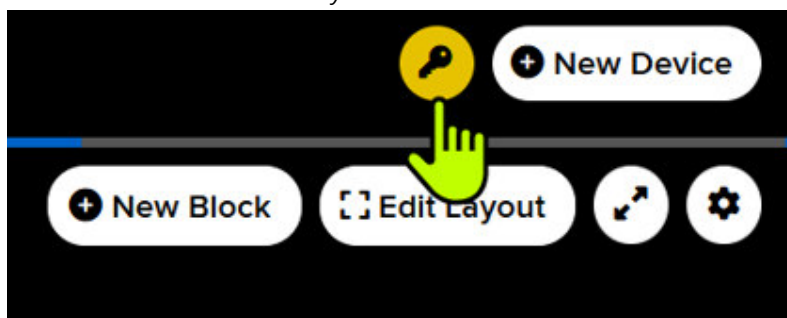
[credential.py](#)

```

ssid = 'Your Wifi Router SSID'
password = 'Your Wifi Router Password'
adafruit_username = b'Your Adafruit UserID'
adafruit_IO_key = b'Your Adafruit IO Key'

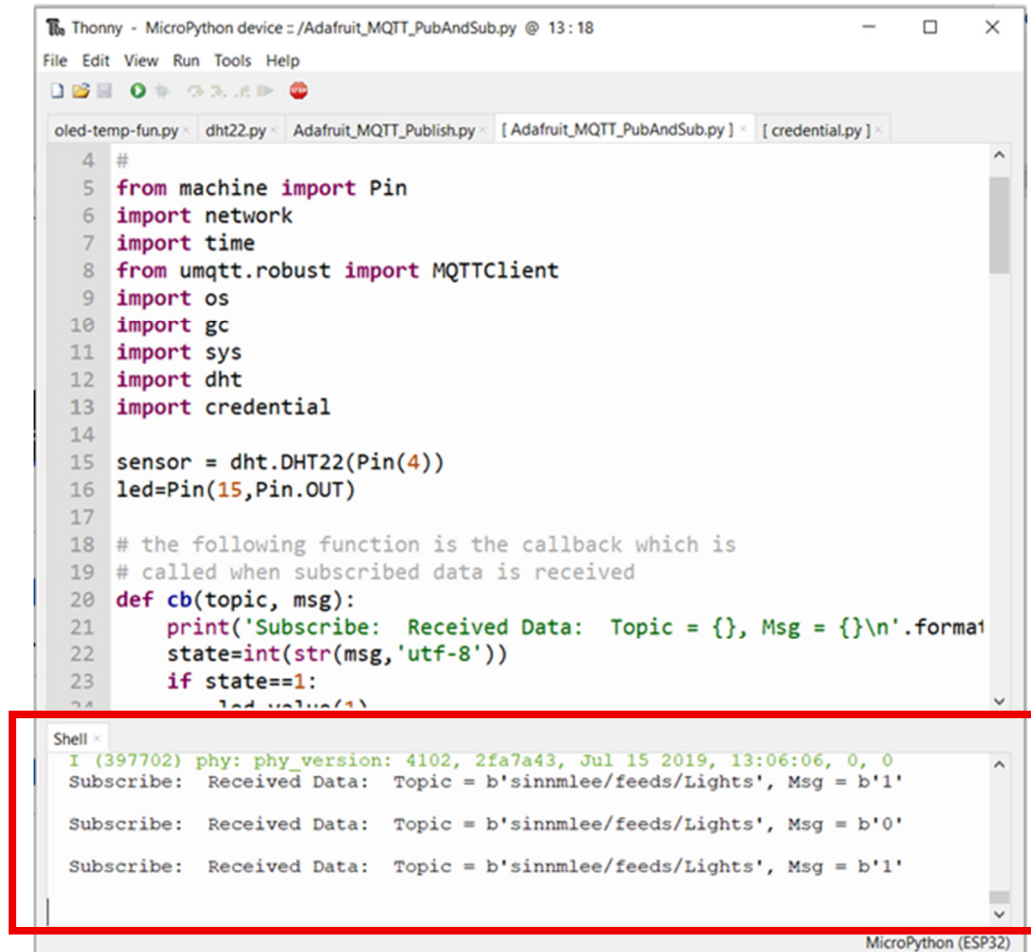
```

- r) You can find the Adafruit IO Key under "MY KEY"



s) Following is the result view in Thonny IDE and Adafruit IO Dashboard:

Using Thonny IDE:



The screenshot shows the Thonny IDE interface with a MicroPython device connected. The main editor displays a script named `Adafruit_MQTT_PubAndSub.py`. The script imports necessary modules and sets up an MQTT client to subscribe to a topic. The Shell window at the bottom shows the execution output, which includes the MQTT client version and three received messages from the topic `b'sinnmlee/feeds/Lights'` with messages `b'1'`, `b'0'`, and `b'1'`.

```

4 #
5 from machine import Pin
6 import network
7 import time
8 from umqtt.robust import MQTTClient
9 import os
10 import gc
11 import sys
12 import dht
13 import credential
14
15 sensor = dht.DHT22(Pin(4))
16 led=Pin(15,Pin.OUT)
17
18 # the following function is the callback which is
19 # called when subscribed data is received
20 def cb(topic, msg):
21     print('Subscribe: Received Data: Topic = {}, Msg = {}'.format(topic, msg))
22     state=int(str(msg,'utf-8'))
23     if state==1:
24         led.value(1)

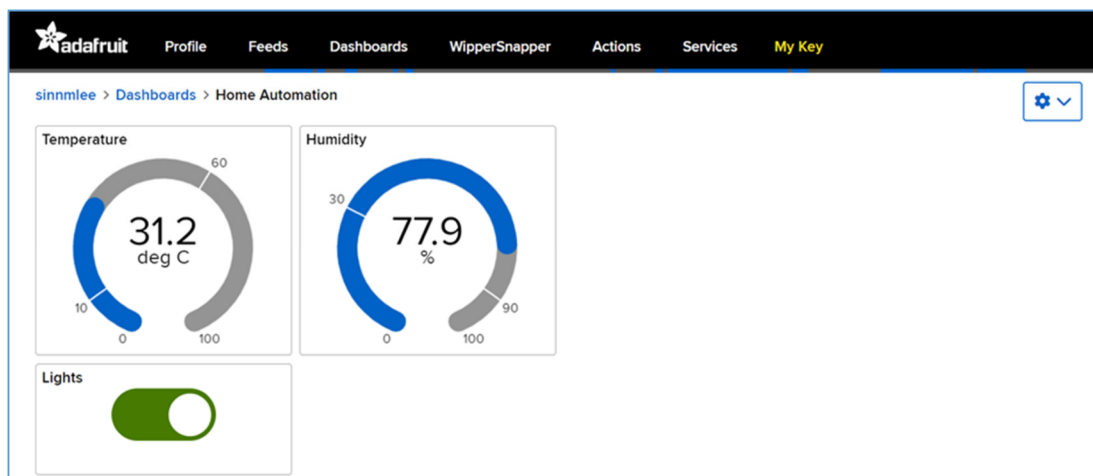
```

```

Shell x
I (397702) phy: phy_version: 4102, 2fa7a43, Jul 15 2019, 13:06:06, 0, 0
Subscribe: Received Data: Topic = b'sinnmlee/feeds/Lights', Msg = b'1'
Subscribe: Received Data: Topic = b'sinnmlee/feeds/Lights', Msg = b'0'
Subscribe: Received Data: Topic = b'sinnmlee/feeds/Lights', Msg = b'1'

```

Adafruit.IO Dashboard:



3. Reflection Questions

- 1) Identify the code sections responsible for MQTT publish and MQTT subscribe.
- 2) Explain the purpose of the callback function `cb(topic, msg)`.
- 3) Explain how the incoming Adafruit IO message is used to control the LED.
- 4) Why must the feed name in the code match the feed name in the Adafruit IO dashboard?
- 5) Why is the `credential.py` file used in this program?
- 6) Why does the program send sensor data every 10 seconds instead of continuously?
- 7) Why must `client.check_msg()` run repeatedly inside the main loop?

4. Summary

In this laboratory exercise, students learned how to use Adafruit IO with ESP32 and MQTT communication to upload DHT22 sensor data and display it on an IoT dashboard. Students also learned how to remotely control an LED through MQTT subscribe communication and gained hands-on experience with cloud-based IoT applications using MicroPython.

- THE END -